

Adversarial Attacks on Neural Networks: A Game Theory Approach

Bachelor's Thesis Defense

Guillem Olivart Garrofé

October 6, 2026

Facultat de Matemàtiques i Estadística - Universitat Politècnica de Catalunya

Introduction and Context

Medical Imaging

Transforming critical sectors:

- **Healthcare:** Spotting tumors in X-rays and MRIs earlier than human doctors.
- **Daily Applications:** Powering self-driving cars and biometric security systems.

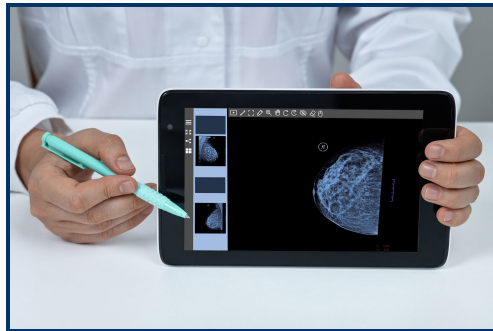


Figure 1: Anomaly detection in lung radiography.

The mathematical catch:

- **Adversarial Attacks:** Adding imperceptible noise to force a confident mistake.

Adversarial Example

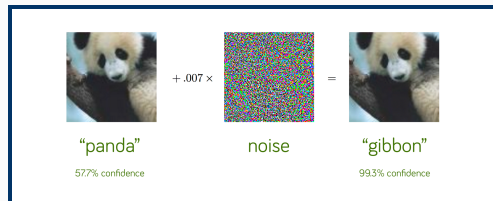


Figure 2: Perturbation leading to misclassification (Goodfellow et al.)

Context: Game Theory Framework

Formalizing the arms race:

- **Attacker:** Maximizes error via strategic noise.
- **Defender:** Strives for accurate classification.
- **The Model:** A two-player zero-sum conflict.
- **Goal:** Finding the **Nash Equilibrium** to prove theoretical limits.

Zero-Sum Payoff Matrix

	Attack 1	Attack 2
Defend 1	+1	+2
Defend 2	-1	+3

Figure 3: Defender's payoffs (Attacker seeks to minimize these values).

Fundamentals of Neural Networks

Mathematical Framework:

An **Artificial Neural Network** is a specific instance of a parameterized function $f_{\theta} \in \mathcal{F} \subseteq \{f \mid f : \mathcal{X} \rightarrow \mathcal{Y}\}$ mapping an input space $\mathcal{X} \subseteq \mathbb{R}^n$ to an output space $\mathcal{Y} \subseteq \mathbb{R}^m$, with learnable parameters $\theta \in \Theta \subseteq \mathbb{R}^d$:

$$f_{\theta}(\mathbf{x}) = \mathbf{y} \in \mathcal{Y}, \quad \text{for } \mathbf{x} \in \mathcal{X}.$$

Task Categorization (based on $\mathcal{Y}$):

- **Regression:** Continuous target ($\mathcal{Y} = \mathbb{R}^m$).
- **Classification:** Discrete classes $\mathcal{C} = \{C_1, \dots, C_m\}$.

Relaxed into the **Probability Simplex** for gradient optimization:

$$\mathcal{Y} = \Delta^{m-1} = \left\{ \mathbf{p} \in \mathbb{R}^m \mid \sum_{i=1}^m p_i = 1, p_i \geq 0 \right\}.$$

Neural Networks: The Artificial Neuron

Artificial Neuron

A function mapping an input vector $\mathbf{x} \in \mathbb{R}^n$ to a scalar output $a \in \mathbb{R}$ via an affine transformation and non-linear activation function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$:

$$a = \sigma(\mathbf{w}^T \mathbf{x} + b),$$

where $\mathbf{w} \in \mathbb{R}^n$ is the weight vector and $b \in \mathbb{R}$ is the bias.

Common Activations (σ)

Purpose: Prevents linear collapse.

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$.
- **ReLU:** $\text{ReLU}(z) = \max(0, z)$.

Neuron Architecture

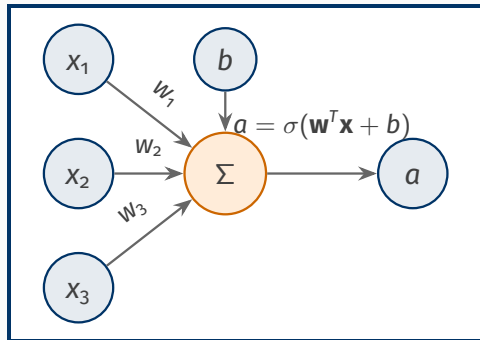
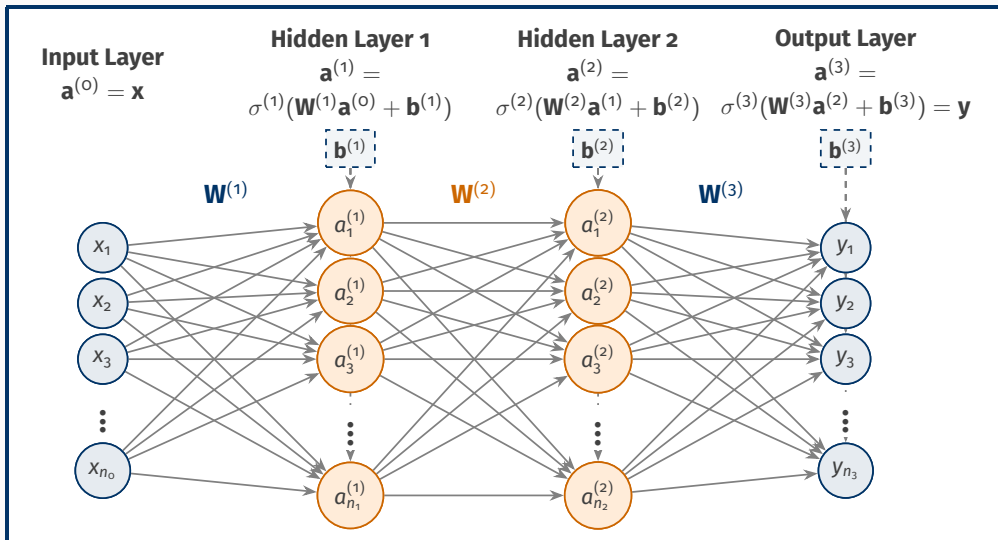


Figure 4: Perceptron with weights, bias, and activation.

Neural Networks: Multilayer Perceptron (MLP) Architecture



Neural Networks: Fully Connected Layer

Layer Parameters

A layer maps an input $\mathbf{x} \in \mathbb{R}^n$ to an output $\mathbf{a} \in \mathbb{R}^m$ via a weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ and a bias vector $\mathbf{b} \in \mathbb{R}^m$:

$$\mathbf{W} = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_m^T \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}.$$

Activation Output

The output vector $\mathbf{a}$ is computed by applying σ component-wise:

$$\mathbf{a} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where each individual component $a_i \in \mathbb{R}$ is explicitly given by:

$$a_i = \sigma_i(\mathbf{w}_i^T \mathbf{x} + b_i).$$

Neural Networks: Multilayer Perceptron

Network Composition

Sequential mapping $f_{\theta} : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}$ of L layers:

$$f_{\theta}(\mathbf{x}) = (\mathcal{A}^{(L)} \circ \dots \circ \mathcal{A}^{(1)})(\mathbf{x}).$$

Forward pass for $\mathcal{A}^{(l)} : \mathbb{R}^{n_{l-1}} \rightarrow \mathbb{R}^{n_l}$ (with $\mathbf{a}^{(0)} = \mathbf{x}$):

$$\mathbf{a}^{(l)} = \mathcal{A}^{(l)}(\mathbf{a}^{(l-1)}) = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$.

Softmax Output

The final raw scores (logits) $\mathbf{z} = \mathbf{a}^{(L)} \in \mathbb{R}^{n_L}$ are mapped onto a valid probability distribution:

$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{n_L} e^{z_j}}.$$

Note: Ensures output values are strictly positive and satisfy $\sum_{i=1}^{n_L} \text{Softmax}(\mathbf{z})_i = 1$.

Neural Networks: Loss Function & Empirical Risk

Loss Function ($\mathcal{L}$)

Mapping to evaluate the discrepancy between prediction $\hat{\mathbf{y}} = f_{\theta}(\mathbf{x}) \in \mathcal{Y}$ and true target $\mathbf{y} \in \mathcal{Y}$:

$$\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}.$$

Cross-Entropy Loss

For a true one-hot target $\mathbf{y} \in \{0, 1\}^m$ and a predicted probability distribution $\hat{\mathbf{y}} \in \Delta^{m-1}$:

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{i=1}^m y_i \log(\hat{y}_i).$$

Empirical Risk ($\mathcal{J}(\theta)$)

Average loss over a training dataset

$\mathcal{D} = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$ of N i.i.d. samples:

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(f_{\theta}(\mathbf{x}^{(n)}), \mathbf{y}^{(n)}).$$

- $\theta = \{(\mathbf{W}^{(l)}, \mathbf{b}^{(l)})\}_{l=1}^L \in \Theta \subseteq \mathbb{R}^d$: Model parameters.
- $\mathcal{D} \in (\mathcal{X} \times \mathcal{Y})^N$: Dataset ($\mathbf{x}^{(n)} \in \mathcal{X}, \mathbf{y}^{(n)} \in \mathcal{Y}$).
- $\mathcal{J} : \Theta \rightarrow \mathbb{R}$: Empirical risk mapping.

Neural Networks: Gradient Descent

Global Optimization

Find the optimal parameter configuration

$\theta^* \in \Theta \subseteq \mathbb{R}^d$ that minimizes the empirical risk $\mathcal{J}(\theta)$:

$$\theta^* = \arg \min_{\theta \in \Theta} \mathcal{J}(\theta).$$

Gradient Descent Update

Iteratively update parameters $\theta^{(t)} \in \Theta \subseteq \mathbb{R}^d$ opposite to the gradient $\nabla_{\theta} \mathcal{J} = \left(\frac{\partial \mathcal{J}}{\partial \theta_1}, \dots, \frac{\partial \mathcal{J}}{\partial \theta_d} \right)^{\top} \in \mathbb{R}^d$, scaled by learning rate $\eta \in \mathbb{R}^+$:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{J}(\theta^{(t)}).$$

Loss Surface Navigation

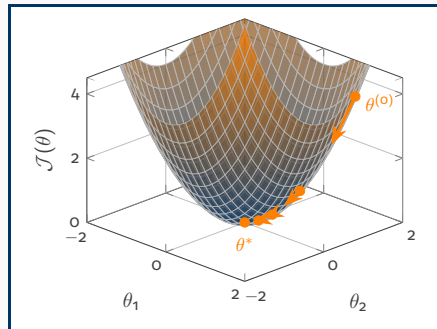


Figure 5: Iterative convergence toward the global minimum.

Neural Networks: Backpropagation

The Chain Rule Expansion

Since $\mathcal{J}(\theta)$ is a sum of individual sample losses, we focus our analysis on evaluating the gradient for a single loss $\mathcal{L}$ via the multivariable chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{W}^{(l)}},$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}.$$

- $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}}$: Error propagated from the next layer.
- $\frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}}$: Jacobian of the activation function.
- $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{W}^{(l)}}$: Local derivative with respect to the weights.
- $\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}}$: Local derivative with respect to the bias.

Single Sample Loss Function

$$\mathcal{L} \left(f_{\theta}(\mathbf{a}^{(0)}), \mathbf{y}^{(n)} \right) = \mathcal{L} \left(\mathbf{a}^{(L)}, \mathbf{y}^{(n)} \right),$$

where for $l = 1, \dots, L$:

$$\mathbf{a}^{(l)} = \sigma^{(l)}(\mathbf{z}^{(l)}) \text{ and } \mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}.$$

Computational Graph (Layer l)

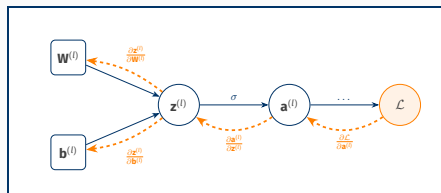


Figure 6: Backward flow of gradients inside a single layer.

Neural Networks: Image Classification

How do computers see images?

Goal: Image classification (assigning a label to an image). We benchmark on standard datasets like **CIFAR-10**.

Spatial Structure: An image is not a flat vector, but a 3D grid/tensor that preserves local spatial topology:

$$\mathbf{I} \in \mathbb{R}^{C_{in} \times H_{in} \times W_{in}}.$$

- $C_{in} = 3$: Color channels (Red, Green, Blue).
- $H_{in} \times W_{in}$: Spatial height and width.

Input Tensor Topology

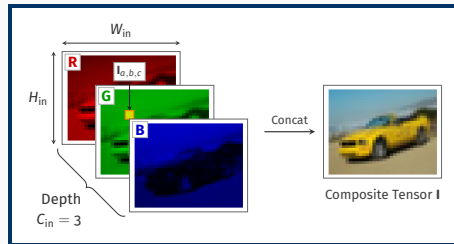


Figure 7: RGB image tensor decomposition.

Neural Networks: CNN Architecture

A **Convolutional Neural Network (CNN)** is a parameterized mapping $f_{\theta} : \mathbb{R}^{C_{in} \times H_{in} \times W_{in}} \rightarrow \mathbb{R}^K$ defined as the composite function:

$$f_{\theta} = f_{\text{head}} \circ f_{\text{feat}},$$

where f_{feat} represents a spatial feature extraction function and f_{head} denotes a classification mapping yielding the output decision vector $\mathbf{y} \in \mathbb{R}^K$.

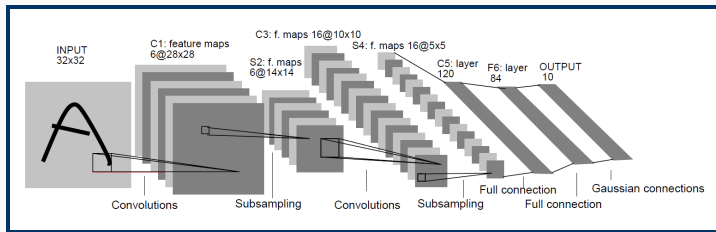


Figure 8: Architecture of LeNet-5: spatial reduction and channel expansion.

Neural Networks: Feature Extractor (f_{feat})

f_{feat} is constructed as a sequence of L computational blocks. Each block $B^{(l)}$ receives the intermediate tensor from the previous stage, starting with the raw image $\mathbf{X}^{(0)} = \mathbf{I} \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$.

$$\mathbf{X}^{(l)} = B^{(l)}(\mathbf{X}^{(l-1)}) = \text{Pool}\left(\sigma\left(\text{BN}\left(\text{Conv}(\mathbf{X}^{(l-1)})\right)\right)\right).$$

- **Conv (Linear Convolution):** Extracts spatial features and expands channel depth (C).
- **BN (Batch Normalization):** Normalizes mini-batch activations to zero mean and unit variance.
- σ **(Activation):** Introduces non-linearity (typically **ReLU**).
- **Pool (Spatial Pooling):** Downsamples spatial dimensions ($H \times W$) for hierarchical abstraction.

Neural Networks: The Convolution Kernel

The Conv Operation

This spatial sliding dot product is precisely how the **Conv** step processes tensors inside each block of f_{feat} .

Dimensional Mapping:

- **Input Tensor:** $\mathbf{X} \in \mathbb{R}^{C_{in} \times H_{in} \times W_{in}}$.
- **Filter Bank:** $\mathbf{K} \in \mathbb{R}^{C_{out} \times C_{in} \times F \times F}$.
- **Output Map:** $\mathbf{S} \in \mathbb{R}^{C_{out} \times H_{out} \times W_{out}}$.
- **Bias Vector:** $\mathbf{b} \in \mathbb{R}^{C_{out}}$.

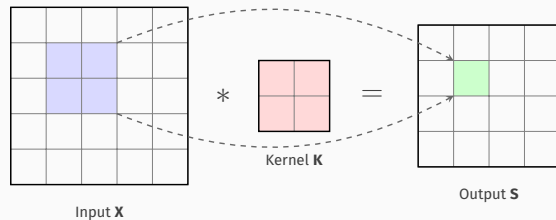


Figure 9: Mechanics of a 2D convolution.

From Features to Final Prediction

1. Flattening Transformation

- **Input:** Tensor $\mathbf{X}^{(L)} \in \mathbb{R}^{C_L \times H_L \times W_L}$.
- Unrolls into 1D vector $\mathbf{v} \in \mathbb{R}^D$
($D = C_L \cdot H_L \cdot W_L$).

Coordinate Mapping:

$$\mathbf{v}_d = \text{Flatten}(\mathbf{X}^{(L)})_d = \mathbf{X}_{c,i,j}^{(L)} \quad \text{for} \quad d = c \cdot (H_L W_L) + i \cdot W_L + j.$$

2. Multilayer Perceptron

- Feeds vector $\mathbf{v} \in \mathbb{R}^D$ into FC layers.
- **Output:** Decision vector $\hat{\mathbf{y}} \in \mathbb{R}^K$.

$$\hat{\mathbf{y}} = \text{MLP}(\mathbf{v}).$$

Game Theory

Strategic Form Game

A game involving simultaneous decision-making is defined as a tuple $\Gamma = (N, S, U)$, where:

- $N = \{1, 2, \dots, n\}$: Finite set of **Players**.
- $S = S_1 \times \dots \times S_n$: **Strategy Space**, where $S_i \in \Delta(S_i)$ is the set of all available strategies for player i .
- $U = (u_1, \dots, u_n)$: Profile of **Utility Functions** $u_i : S \rightarrow \mathbb{R}$.

Game Theory: Pure & Mixed Strategies

Expected Utility

Probability-weighted payoff for player i under mixed profile $\sigma = (\sigma_1, \sigma_2) \in \Delta(S_1) \times \Delta(S_2)$:

$$U_i(\sigma) = \sum_{s_1 \in S_1} \sum_{s_2 \in S_2} \sigma_1(s_1) \sigma_2(s_2) u_i(s_1, s_2).$$

Strictly Competitive (Zero-Sum) Game

Let $\Gamma = \langle \{1, 2\}, S_1 \times S_2, (u_1, u_2) \rangle$ be a two-player normal-form game. Γ is **zero-sum** if:

$$\forall (s_1, s_2) \in S_1 \times S_2, \quad u_1(s_1, s_2) + u_2(s_1, s_2) = 0.$$

- Defining a single objective $u \equiv u_1 \equiv -u_2$ ($u : S_1 \times S_2 \rightarrow \mathbb{R}$), Player 1 is the **maximizer** and Player 2 the **minimizer**.

Game Theory: Nash Equilibrium & Saddle Point

Nash Equilibrium

A strategy profile σ^* is a **Nash Equilibrium** if, for every player i and strategy $\sigma'_i \in \Delta(S_i)$:

$$\mathcal{U}_i(\sigma^*) \geq \mathcal{U}_i(\sigma'_i, \sigma_{-i}^*),$$

where $(\sigma'_i, \sigma_{-i}^*)$ denotes σ^* with i playing σ'_i instead.

Minimax Saddle Point

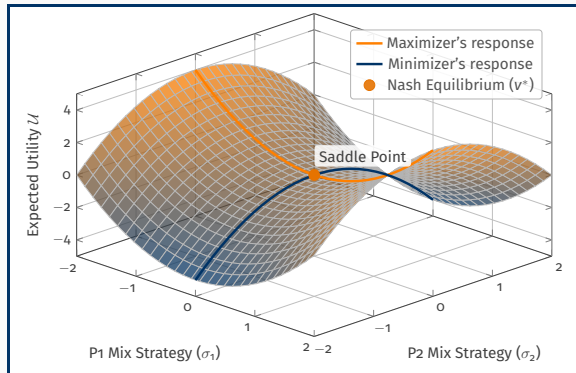


Figure 10: Geometric visualization locking the value v^* .

John von Neumann's Minimax Theorem

For any finite, two-player zero-sum game defined by utility function $\mathcal{U}$, there exists a unique scalar $v^* \in \mathbb{R}$ (the **Value of the Game**) such that:

$$\max_{\sigma_1 \in \Delta(S_1)} \min_{\sigma_2 \in \Delta(S_2)} \mathcal{U}(\sigma_1, \sigma_2) = \min_{\sigma_2 \in \Delta(S_2)} \max_{\sigma_1 \in \Delta(S_1)} \mathcal{U}(\sigma_1, \sigma_2) = v^*.$$

General Methodology

General Methodology: The Architectural Game

Architectural Game Formulation

Architectural ablation is modeled as a **finite, normal-form zero-sum game**:

$$\mathcal{G} = \langle \mathcal{N}, S, \mathcal{U} \rangle.$$

Players ($\mathcal{N} = \{1, 2\}$)

- **Player 1 (Defender):**
Target classifier f_θ .
- **Player 2 (Attacker):**
Generative model G_ϕ .

Strategy Spaces ($S = S_1 \times S_2$)

- $s_1 \in S_1$: **Defensive architecture**
(classifier topology).
- $s_2 \in S_2$: **Generative architecture**
(attacker topology).

Zero-Sum Payoff Structure

$$\mathcal{U} = \{U_1, U_2\}, \quad \text{where} \quad U_2(s_1, s_2) = -U_1(s_1, s_2).$$

General Methodology: Continuous Parameter Training

Continuous Co-Adaptation Phase (Training)

For any strategy pair $(s_1, s_2) \in S_1 \times S_2$, agents engage in **alternating parameter optimization**:

- **Models:** Classifier f_θ ($\theta \in \Theta_{s_1}$) vs. Generator G_ϕ ($\phi \in \Phi_{s_2}$, synthesizing $\delta_\phi(\mathbf{x})$).
- **Dynamics:** Train over $(\mathbf{x}, y) \sim \mathcal{D}_{\text{train}}$ via continuous, differentiable $\mathcal{L}_{\text{CE}}$.
- **Outcome:** Reach optimal finalized weights $(\theta^*, \phi^*) \in \Theta_{s_1} \times \Phi_{s_2}$.

Alternating Optimization Objectives

$$\text{Attacker Step: } \phi^* = \arg \max_{\phi \in \Phi_{s_2}} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{\text{train}}} \left[\mathcal{L}_{\text{CE}} \left(f_\theta(\mathbf{x} + \delta_\phi(\mathbf{x})), y \right) \right]$$

$$\text{Defender Step: } \theta^* = \arg \min_{\theta \in \Theta_{s_1}} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{\text{train}}} \left[\mathcal{L}_{\text{CE}} \left(f_\theta(\mathbf{x} + \delta_\phi(\mathbf{x})), y \right) \right]$$

Discrete Game Evaluation (Payoff Population)

For the finalized weights (θ^*, ϕ^*) , the true strategic value of the architectures is quantified:

- **Data:** Evaluated over a strictly unseen validation holdout $(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{val}}$.
- **Metric:** Generic functional $\mathcal{L}_{\text{eval}}$ (e.g., 0-1 loss, bounded accuracy, custom risk).
- **Outcome:** Yields standard adversarial accuracy bounded within $\mathcal{U}_1 \in [0\%, 100\%]$ to populate the payoff matrix.

Empirical Utility for the Defender ($\mathcal{U}_1$)

$$\mathcal{U}_1(s_1, s_2) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{\text{val}}} \left[\mathbf{1} \left(\arg \max f_{\theta^*}(\mathbf{x} + \delta_{\phi^*}(\mathbf{x})) = y \right) \right].$$

General Methodology: Empirical Payoff Matrix Example

	$s_{2,1}$	$s_{2,2}$	$s_{2,3}$	$s_{2,4}$
$s_{1,1}$	82%	64%	71%	55%
$s_{1,2}$	45%	78%	60%	42%
$s_{1,3}$	68%	52%	75%	61%
$s_{1,4}$	59%	81%	49%	70%

$$\mathcal{M} \in \mathbb{R}^{4 \times 4}, \quad \text{where} \quad \mathcal{M}_{i,j} = \mathcal{U}_1(s_{1,i}, s_{2,j}).$$

Empirical Implementation and Tournament Design

Tournament Design: CIFAR-10 Environment

Evaluation Environment

Evaluated on CIFAR-10 under **visually stealthy** perturbations with input space $\mathcal{X} = [0, 1]^{3 \times 32 \times 32}$ and output label space:

$$\mathcal{Y} = \{\text{airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck}\}.$$

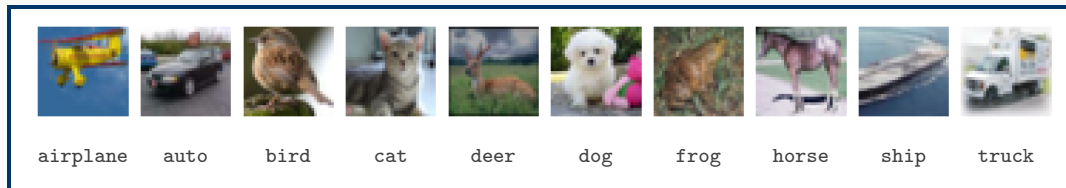


Figure 11: Representative sample across all ten CIFAR-10 semantic categories.

Tournament Design: Shared Architectural Strategy Space ($S_1 \approx S_2$)

CNN Architectural Strategy Formulation

Defender (f_θ) and Attacker (G_ϕ) share a symmetric, discrete structural search space:

1. Depth Strategy (K)

Number of sequential convolutional blocks:

$$K \in \{1, 2, 3, 4\}.$$

2. Width Strategy (C_{base})

Base channel capacities space:

$$C(K) \subset \{2, 8, 16, 32, 64, 128\}.$$

3. Normalization Strategy ($\mathcal{N}$)

Internal statistical standardization operators:

$$\mathcal{N} = \{\text{Id}, \text{BN}, \text{IN}\}.$$

4. Spatial Regularization ($\mathcal{R}$)

DropBlock2d (3×3) parameterized by drop rate γ :

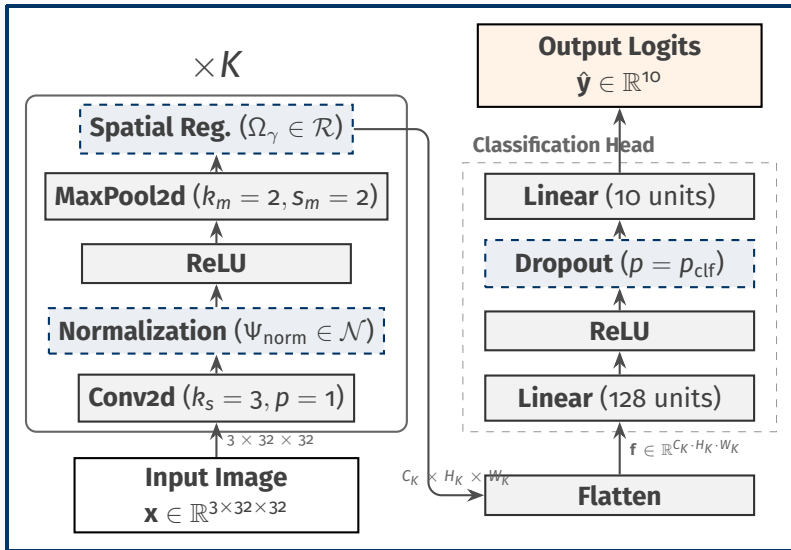
$$\mathcal{R} = \{\Omega_\gamma \mid \gamma \in \{0.0, 0.25, 0.5\}\}.$$

Tournament Design: Defender Strategy Space (S_1)

Table 1: Discrete strategy space for the Defender (S_1).

Strategy (S_1)	Depth (K)	Width (C_{base})	Norm (Ψ_{norm})	Reg (Ω_γ)
D10	4	64	<i>BN</i>	0.0
D9	4	32	<i>BN</i>	0.0
D8	3	64	<i>BN</i>	0.0
D7	3	32	<i>BN</i>	0.0
D6	2	64	<i>BN</i>	0.0
D5	2	32	<i>BN</i>	0.0
D4	2	16	<i>BN</i>	0.0
D3	1	64	<i>BN</i>	0.0
D2	1	32	<i>BN</i>	0.0
D1	1	16	<i>BN</i>	0.0

Tournament Design: Defender Architecture

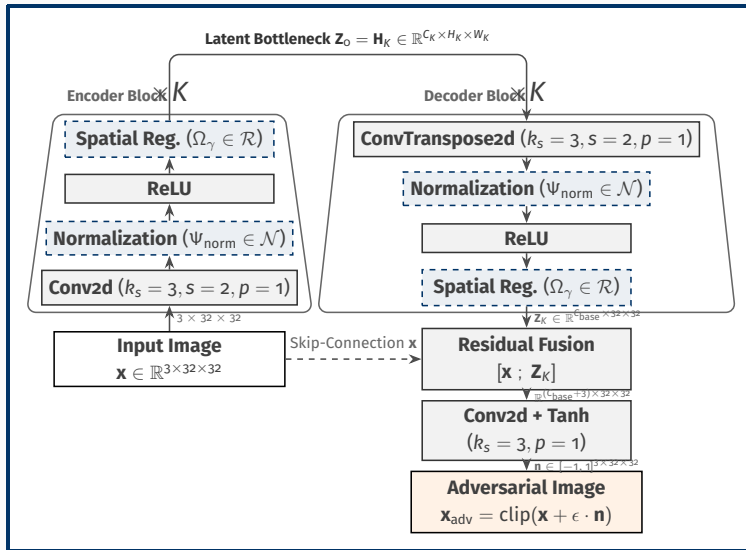


Tournament Design: Attacker Strategy Space (S_2)

Table 2: Discrete strategy space for the Attacker (S_2).

Strategy (S_2)	Depth (K)	Width (C_{base})	Norm (Ψ_{norm})	Reg (Ω_γ)
A10	4	32	<i>BN</i>	0.0
A9	3	64	<i>BN</i>	0.0
A8	3	32	<i>BN</i>	0.0
A7	3	16	<i>BN</i>	0.0
A6	2	32	<i>BN</i>	0.0
A5	2	16	<i>BN</i>	0.0
A4	2	8	<i>BN</i>	0.0
A3	1	32	<i>BN</i>	0.0
A2	1	16	<i>BN</i>	0.0
A1	1	8	<i>BN</i>	0.0

Tournament Design: Attacker Architecture



Tournament Design: Training Dynamics (Phase 1)

Phase 1: Classifier Warm-up

Isolated pre-training (10 epochs) on dataset $\mathcal{D}_{\text{train}}$ to optimize parameters $\theta \in \Theta \in \mathbb{R}^{d_1}$ and map clean images $\mathbf{x} \in \mathcal{D}_{\text{train}}$ to logits $\hat{\mathbf{y}} = f_{\theta}(\mathbf{x}) \in \mathbb{R}^{10}$:

$$\min_{\theta} \mathcal{L}_C^{\text{clean}}(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{\text{train}}} [\mathcal{L}_{\text{CE}}(f_{\theta}(\mathbf{x}), \mathbf{y})].$$

Empirical Risk Minimization

Standard ERM via Categorical Cross-Entropy, targeting the true one-hot vector $\mathbf{y} \in \{0, 1\}^{10}$ for true class y :

$$\mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \left(\frac{\exp(\hat{y}_y)}{\sum_{c=1}^{10} \exp(\hat{y}_c)} \right).$$

Tournament Design: Training Dynamics (Phase 2)

Phase 2a: Attacker Maximization

Defender is frozen. Generator parameters $\phi \in \Phi \in \mathbb{R}^{d_2}$ output noise $\delta_\phi = \epsilon \cdot \mathbf{n}$ for $\mathbf{x} \sim \mathcal{D}_{\text{train}}$, generating $\mathbf{x}_{\text{adv}} \in [-1, 1]^{3 \times 32 \times 32}$ and updating **twice** per minibatch:

$$\mathcal{L}_G(\phi) = -\mathcal{L}_{\text{CE}}(f_\theta(\text{clip}(\mathbf{x} + \delta_\phi)), \mathbf{y}).$$

Phase 2b: Defender Minimization

Attacker is frozen. Defender optimizes $\theta \in \Theta \in \mathbb{R}^{d_1}$ via a dual objective on clean ($\mathbf{x} \in \mathcal{D}_{\text{train}}$) and adversarial ($\mathbf{x}_{\text{adv_final}}$) batches to prevent **catastrophic forgetting**:

$$\mathcal{L}_C(\theta) = \frac{\mathcal{L}_{\text{clean}}(\theta) + \mathcal{L}_{\text{adv}}(\theta)}{2}.$$

Tournament Design: Adversarial Attack & Defense Visualizations

(a) Successful Attack: Attacker A10 vs Defender D1

Original Image



Real: frog → Pred: frog

Adversarial Noise (δ)



Strategy: A10

Adversarial Image



Pred vs D1: deer (Wrong)

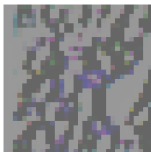
(b) Robust Defense: Attacker A1 vs Defender D10

Original Image



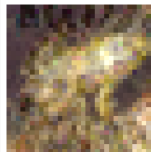
Real: frog → Pred: frog

Adversarial Noise (δ)



Strategy: A1

Adversarial Image



Pred vs D10: frog (Good)

Tournament Design: Empirical Payoff Matrices Construction

Validation Accuracy Matrices

Computed over $\mathcal{D}_{\text{val}}$ for $m = |S_1|$ and $n = |S_2|$ at epoch t :

1. Clean Accuracy ($\mathcal{M}_{\text{clean}}^{(t)} \in \mathbb{R}^{m \times n}$)

$$(\mathcal{M}_{\text{clean}}^{(t)})_{i,j} = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{\text{val}}} \left[\mathbf{1}_{\{f_{\theta_i}^{(t)}(\mathbf{x}) = y\}} \right].$$

2. Adversarial Accuracy ($\mathcal{M}_{\text{adv}}^{(t)} \in \mathbb{R}^{m \times n}$)

$$(\mathcal{M}_{\text{adv}}^{(t)})_{i,j} = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}_{\text{val}}} \left[\mathbf{1}_{\{f_{\theta_i}^{(t)}(\mathbf{x} + \delta_j) = y\}} \right].$$

Game Payoff Formulation

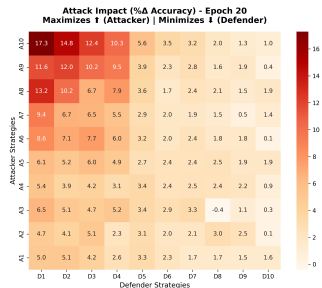
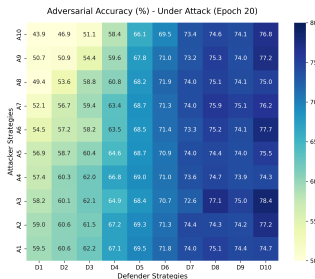
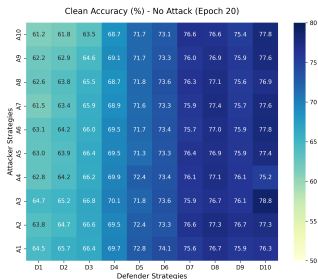
Matrix metrics derived from performance drops:

3. Absolute Delta ($\mathcal{M}_{\Delta}^{(t)} \in \mathbb{R}^{m \times n}$)

$$\mathcal{M}_{\Delta}^{(t)} = \mathcal{M}_{\text{clean}}^{(t)} - \mathcal{M}_{\text{adv}}^{(t)}.$$

Results and Discussion

Results and Discussion: Empirical Payoff Heatmaps (Epoch 20, $\epsilon = 0.1$)

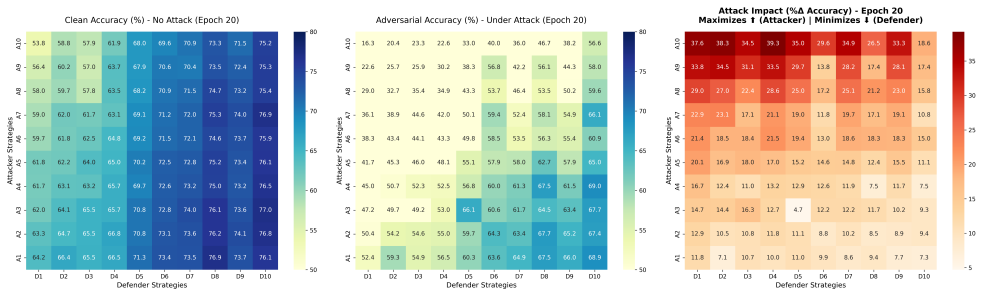


Clean Accuracy ($\mathcal{M}_{\text{clean}}$)

Adversarial Accuracy ($\mathcal{M}_{\text{adv}}$)

Zero-Sum Payoff ($\mathcal{M}_{\Delta}$)

Results and Discussion: Empirical Payoff Heatmaps (Epoch 20, $\epsilon = 0.2$)



Clean Accuracy ($\mathcal{M}_{\text{clean}}$)

Adversarial Accuracy ($\mathcal{M}_{\text{adv}}$)

Zero-Sum Payoff ($\mathcal{M}_{\Delta}$)

Results and Discussion: Computational Resolution via Linear Programming

Nash Equilibrium Computation

Solved per epoch t on $A = \mathcal{M}_{\text{rel}}^{(t)} \in \mathbb{R}^{m \times n}$ via the `highs` dual-simplex solver.

1. Attacker (Primal Maximization)

Finds mixed strategy $q^* \in \Delta(S_2)$ to maximize v :

$$\begin{aligned} \max_{q \in \Delta(S_2), v} \quad & v \\ \text{s.t.} \quad & \sum_{i=1}^n q_i A_{i,j} \geq v, \quad \forall j \\ & \sum_{i=1}^n q_i = 1, \quad q_i \geq 0. \end{aligned}$$

2. Defender (Dual Minimization)

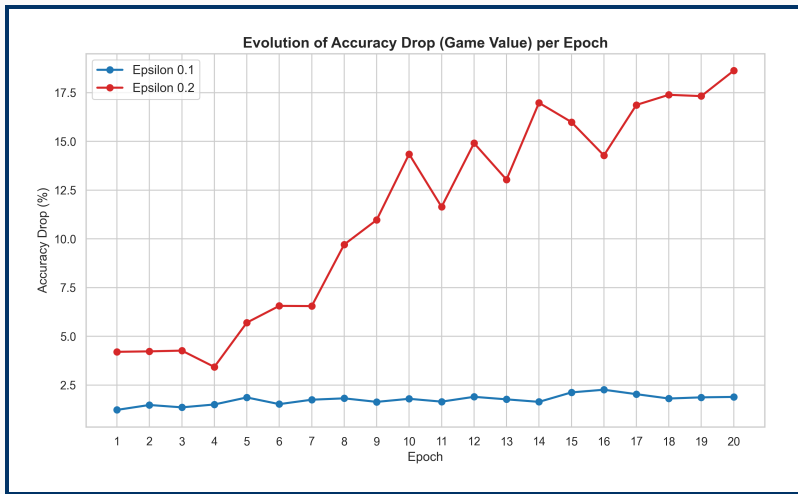
Finds mixed strategy $p^* \in \Delta(S_1)$ to minimize u :

$$\begin{aligned} \min_{p \in \Delta(S_1), u} \quad & u \\ \text{s.t.} \quad & \sum_{j=1}^m p_j A_{i,j} \leq u, \quad \forall i \\ & \sum_{j=1}^m p_j = 1, \quad p_j \geq 0. \end{aligned}$$

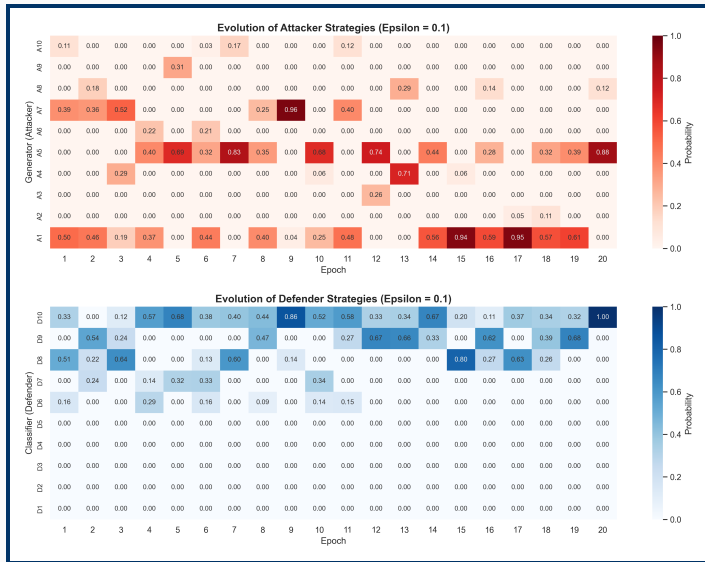
Minimax Theorem (Strong Duality)

Strong duality guarantees convergence to a unique game value: $\mathbf{V}^* = \mathbf{u} = \mathbf{v} = (\mathbf{p}^*)^T \mathbf{A} \mathbf{q}^*$.

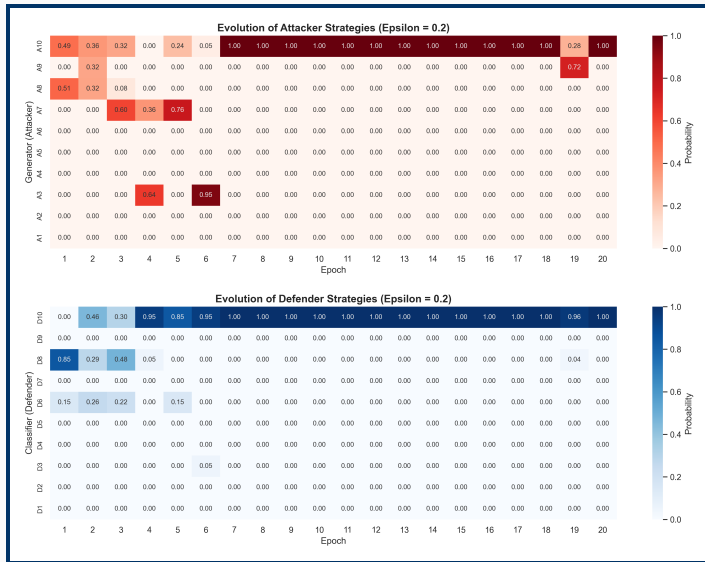
Results and Discussion: Evolution of Game Utility (10×10 Experiments)



Results and Discussion: Evolution of Mixed Strategies ($\epsilon = 0.1$)



Results and Discussion: Evolution of Mixed Strategies ($\epsilon = 0.2$)



Conclusions and Future Work

Key Conclusions

- **Game-Theoretic Model:** Developed a proof-of-concept framework for adversarial co-adaptation as a 2-player zero-sum LP game.
- **Impact of ϵ :** The perturbation bound (ϵ) is a critical parameter that dictates attacker capabilities and fundamentally shapes equilibrium strategies.
- **Strategic Asymmetry:** Defenders prefer simple models; Attackers strictly require regularization.

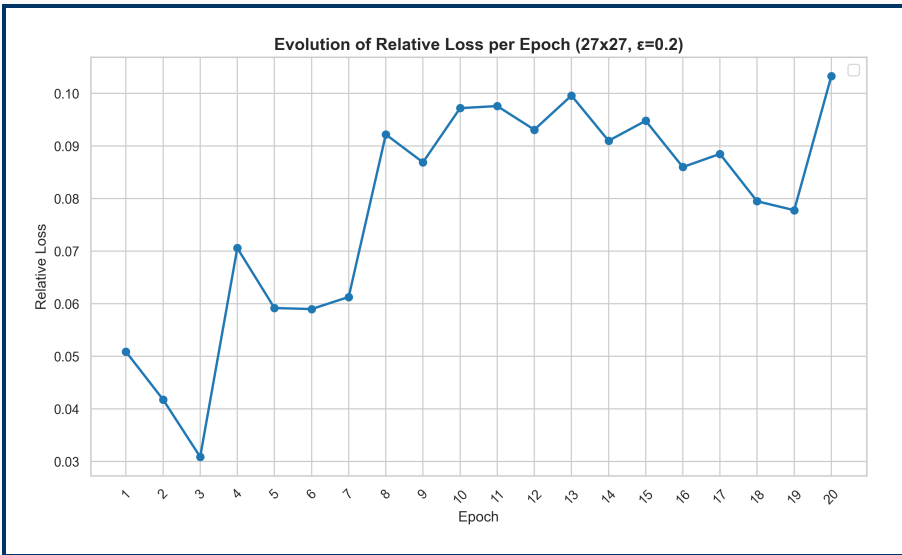
Future Work

- **3-Player Extension:** Add a regulator agent, shifting from LP to non-linear Nash Equilibria.

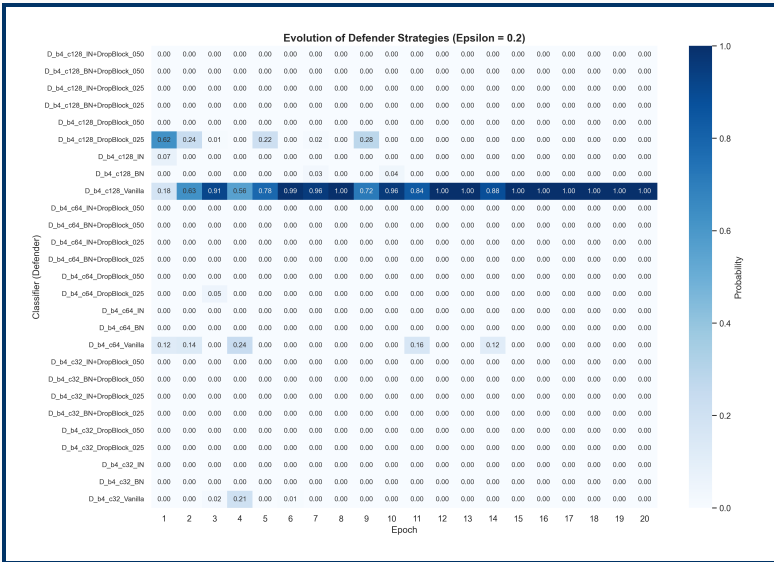
Thank you for your attention!

Questions & Discussion

Backup: Evolution of Game Utility (27×27 , $\epsilon = 0.2$)



Backup: Defender Strategies ($27 \times 27, \epsilon = 0.2$)



Backup: Attacker Strategies ($27 \times 27, \epsilon = 0.2$)

